Introduction to R & Environmental Data

SHM 377 Hazardous Materials Management

Yoni Rodriguez

Table of contents

Virtual Lab 01 3
- Introduction to R & Environmental Data . . . 3

The Monitoring Question 4

The Dataset 4

Why Are We Using R? 5

R and RStudio 5
- What is R? . . . 5
- What is RStudio? . . . 6

Installing R and RStudio 6
- Windows . . . 7
- macOS . . . 7
- Check Your Installation . . . 7
- The RStudio Interface . . . 8

Organizing Your Work 9
- Create a Course Folder . . . 9
- File Naming . . . 9

Create an RStudio Project 10

Organizing an R Script with Comments 11
- What Is a Comment? . . . 11
- Comments Can Also Follow Code . . . 12
- Using Comments as Section Headers . . . 12

Commands and Functions 13
- What Is an R Command? . . . 13
- What Is an R Function? . . . 14

The Console and the Script Have Different Jobs 14

Running Code from Your R Script 16

Your First R Commands 16

Creating an Object 17

Getting the Dataset 17
Download the Dataset . . . 18

Where Is R Looking? 19
Check the Working Directory . . . 19
See What Files R Can Find . . . 20

Set the Working Directory 20

Importing the Dataset 21

Inspect Before You Analyze 22

Look at the First Rows 23

Look at the Last Rows 23

Variable Names 23

Dataset Dimensions 24

Examine the Structure 24

Did R Understand the Date and Time? 25

Check the Measurements 26
Minimum . . . 27
Maximum . . . 27

Unusual Values 27

Describe the Data 28
Mean . . . 28
Median . . . 28
Summary . . . 29

Descriptive Statistics Have Limits 29

Visualizing Concentration Through Time 29

Reading the Plot Function 30
Arguments . . . 30
Why Do We Use Commas? . . . 31
Why Does `date_time` Come First? . . . 31
Positional Arguments . . . 32

Named Arguments . . . 32
What Does `type = "l"` Mean? . . . 33
What Does `xlab` Mean? . . . 33
What Does `ylab` Mean? . . . 33
What Does = Mean Inside the Function? . . . 33

Exporting Your Plot 34

What Should Be in Your R Script? 34

Why Units Matter 36

Look Before Explaining 37

Observation and Interpretation 37

What Can We Conclude? 38

What Can We Not Conclude? 38

What Other Data Could Help? 38

Return to the Original Question 38

The SHM 377 Data Workflow 39

Connecting Data to Hazardous Materials Management 39

Reproducibility 40

You Do Not Need to Memorize Everything 41

What You Accomplished 41

The Main Idea 42

Where We Go Next 42

Virtual Lab 01

Introduction to R & Environmental Data

Today we will use a real environmental monitoring dataset to answer a simple question:

> **What happened to ambient particle concentrations during five days of winter monitoring?**

We will use R to:

- obtain the data

- inspect the data
- check the measurements
- calculate simple summaries
- create a visualization
- interpret what we see

The goal is not to become a programmer today.

The goal is to begin using R as a tool for environmental and hazardous materials data analysis.

The Monitoring Question

We have repeated measurements of airborne particles collected during winter monitoring.

The measurements were collected approximately every two minutes from:

December 27 through December 31, 2019

Our question is:

> **What happened to ambient particle concentrations during these five days?**

Before doing any analysis, we need to understand what the dataset contains.

The Dataset

The teaching dataset contains approximately 3,600 observations.

It contains two variables:

Variable	Meaning
`date_time`	When the measurement was collected
`concentration`	Measured particle concentration

The measurements were collected using a **TSI DustTrak**.

Particle concentration was measured in **milligrams per cubic meter (mg/m³)**.

The monitoring instrument measured particle concentration.

It did not identify the exact source of every particle.

This distinction is important.

> **Measurement does not equal source attribution.**

If concentration increases, we can observe the increase.

We cannot automatically conclude what caused it.

Why Are We Using R?

Environmental and hazardous materials work often produces large amounts of data.

Examples include:

- air monitoring
- water sampling
- soil sampling
- surface sampling
- direct reading instrument data
- meteorological measurements
- exposure measurements

R gives us a way to organize, inspect, analyze, visualize, and communicate those data.

Another major advantage is reproducibility.

Instead of clicking through a series of menus and trying to remember what we did, we can save the commands used for the analysis.

R and RStudio

We will use two pieces of software in this course: **R** and **RStudio**.

They work together, but they are not the same thing.

> **R is the program that performs the analysis. RStudio is the interface we use to work with R.**

Both are available for free.

What is R?

R is a programming language and analytical environment used for:

- data analysis
- statistics
- visualization
- reproducible research

When we ask R to calculate a mean, summarize a dataset, or create a graph, **R is the software actually performing those operations**.

R is free software and is available for Windows, macOS, and Linux.

Download R from the R Project

What is RStudio?

RStudio is an integrated development environment, or IDE, that provides a convenient interface for working with R.

Instead of working with R through a basic command window, RStudio gives us an organized workspace where we can:

- write and save code
- run R commands
- view our data
- create and view graphs
- navigate files
- access help and documentation

RStudio Desktop is available in a free open source edition.

Download RStudio Desktop from Posit

Installing R and RStudio

Before we begin working with data, we need both **R** and **RStudio Desktop** installed on your computer.

Both are free.

> **!** Installation order matters
>
> Install the software in this order:
>
> 1. **Install R**
> 2. **Install RStudio Desktop**
> 3. **Open RStudio**
> 4. **Confirm that RStudio can find and run R**
>
> RStudio provides the interface, but R performs the computations.

Windows

1. Go to the R Project.
2. Select **CRAN**.
3. Choose a CRAN mirror.
4. Select **Download R for Windows**.
5. Download the current version of R.
6. Run the installer and follow the installation prompts.
7. Go to the RStudio Desktop download page.
8. Download the free version of RStudio Desktop for Windows.
9. Run the RStudio installer.
10. Open RStudio.

macOS

The installation process on macOS depends on your version of macOS and your Mac's processor.

1. Go to the R Project.
2. Select **CRAN**.
3. Choose a CRAN mirror.
4. Select **Download R for macOS**.
5. Read the available installer descriptions carefully.
6. Download the version appropriate for your Mac.
7. Install R.
8. Go to the RStudio Desktop download page.
9. Download the free version of RStudio Desktop for macOS.
10. Install RStudio.
11. Open RStudio.

> ⚠ macOS users
>
> Do not guess which installer you need.
> Different Mac computers may use different processors. We will verify your Mac and select the appropriate installer before continuing if you are unsure.

Check Your Installation

Once RStudio opens, locate the **Console** pane.

You should see information about the version of R that is running followed by the `>` prompt.

At the prompt, enter:

```
2 + 2
```

If R returns:

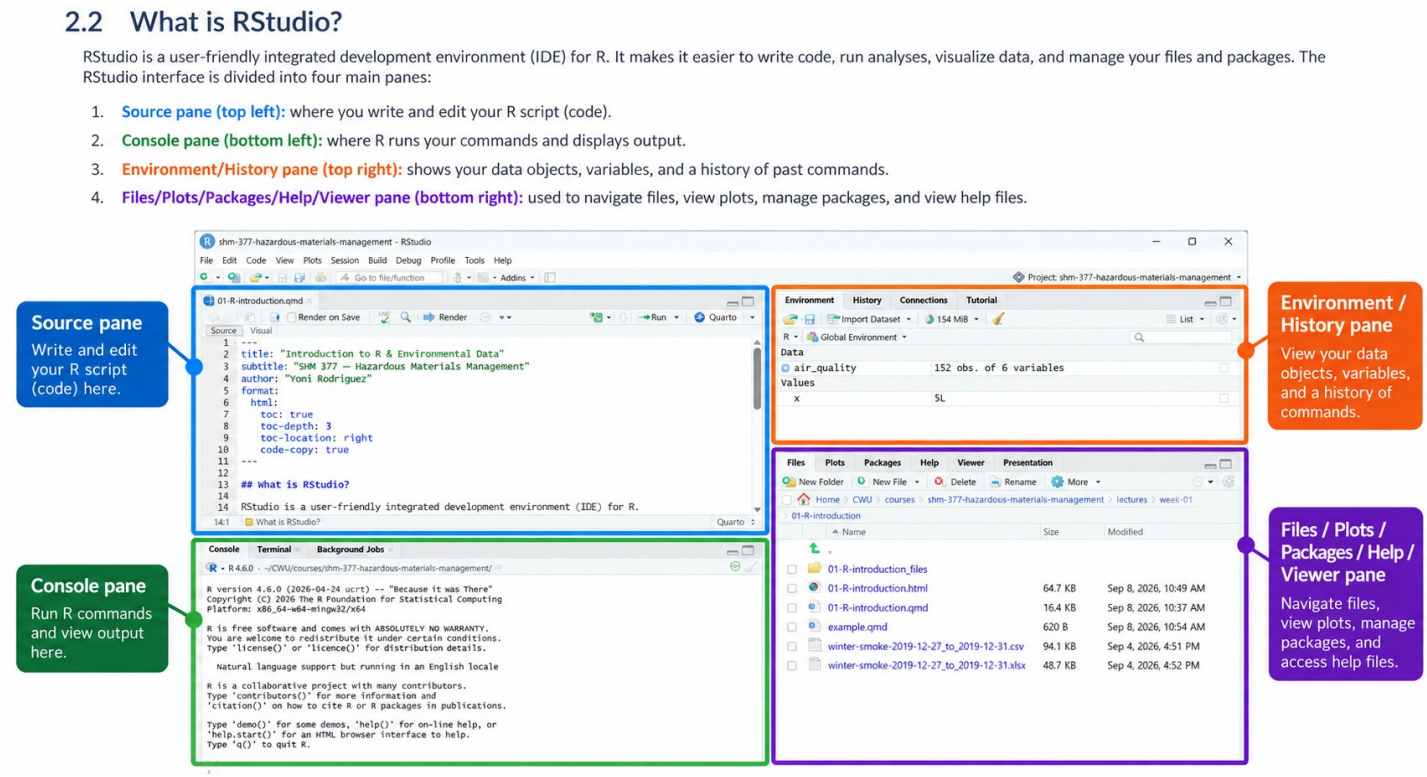
```
[1] 4
```

R and RStudio are communicating correctly.

> **Need more help?**
>
> If you have trouble installing R or RStudio, use the official installation resources:
>
> - R Project
> - RStudio Desktop Download
> - RStudio Desktop Installation Documentation
>
> If you are unsure which version to install, especially on macOS, ask for help before continuing.

The RStudio Interface

You will commonly see four areas in RStudio:



One important habit for this course is:

> **Write your commands in the Source pane rather than relying only on the Console.**

The Source pane allows us to save our code and creates a record of what we did.

Organizing Your Work

Before downloading our dataset, we need to decide where our files will live.

Good file organization makes working with data much easier. Many problems in R happen because files are saved in different locations and R cannot find them.

For this course, we will keep files associated with the same project together.

Create a Course Folder

Create a folder on your computer called:

```
SHM-377
```

Inside that folder, create a folder for today's lab:

```
SHM-377/
   virtual-lab-01/
```

We will save everything for today's analysis inside `virtual-lab-01`.

By the end of the lab, your folder may look something like this:

```
SHM-377/
   virtual-lab-01/
       winter-smoke-2019-12-27_to_2019-12-31.csv
       virtual-lab-01.R
```

> **!** Keep your project files together
>
> Do not save the dataset in one location, your R script somewhere else, and your other project files in another folder.
> Keeping related files together makes your analysis easier to reproduce and reduces problems caused by missing files or incorrect file paths.

File Naming

Use file names that tell you what the file contains.

Good:

```
winter-smoke-2019-12-27_to_2019-12-31.csv
virtual-lab-01.R
```

Avoid names such as:

```
data.csv
data-final.csv
data-final-2.csv
stuff.csv
```

Clear file names become increasingly important as projects grow.

Create an RStudio Project

Now that we have organized our files, we will create an **RStudio Project**.

An RStudio Project gives RStudio a defined home for the work associated with a project.

For SHM 377, this helps us keep our:

- data
- R scripts
- figures
- reports
- other course files

organized together.

In RStudio:

1. Select **File**.
2. Select **New Project**.
3. Select **Existing Directory**.
4. Navigate to your `SHM-377` folder.
5. Select the folder.
6. Select **Create Project**.

RStudio will reopen using that folder as the project.

You should now see the project name in the upper right corner of RStudio.

> A simple rule for this course
>
> **One course project, organized folders within that project.**
> Your RStudio Project gives the course a home.
> Folders inside the project help organize individual labs and analyses.

Organizing an R Script with Comments

Before we begin adding R commands to our script, let's look at how we can organize and document our work.

A simple beginning for your script might look like this:

```
# Virtual Lab 01
# Your Name
# Date
```

Notice that each line begins with:

```
#
```

In an R script, the # symbol begins a **comment**.

What Is a Comment?

A comment is text written for the person reading the code.

R does not run anything that appears after # on that line.

For example:

```
# Import the dataset
```

is a comment.

R sees the # and ignores the rest of that line when the script is run.

Comments allow us to explain what different parts of our code are doing.

For example:

```
# Import the dataset

winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)

# Convert the date and time variable

winter_smoke$date_time <- as.POSIXct(
  winter_smoke$date_time
)
```

The comments help us understand the purpose of each section, but only the R commands are executed.

Comments Can Also Follow Code

The # does not have to appear at the beginning of a line.

For example:

```
x <- 5  # Store the value 5 in x
```

R runs:

```
x <- 5
```

but ignores:

```
# Store the value 5 in x
```

Everything after # on that line is treated as a comment.

> **!** R does not run comments
>
> When R encounters #, it treats everything after it on that line as a comment.
>
> ```
> # This entire line is a comment
>
> x <- 5  # R runs x <- 5 but ignores this explanation
> ```
>
> Comments are for people reading the script, not instructions for R to execute.

Using Comments as Section Headers

We can also use comments to organize our script into sections.

For example:

```
# Virtual Lab 01
# Your Name
# Date


# Import Data

winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)


# Convert Date and Time
```

```
winter_smoke$date_time <- as.POSIXct(
  winter_smoke$date_time
)


# Create Visualization

plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

The lines beginning with # act as labels that make the script easier to read.

As our analyses become longer, these comments will help us quickly identify different parts of our work.

> Write code for your future self
>
> A good R script should not only run correctly.
> It should also help you understand what you did when you return to the analysis later.
> Use comments to explain and organize your work.

Commands and Functions

Before we continue, we need to distinguish two terms that we will use throughout this course.

What Is an R Command?

A **command** is any instruction that we give R.

For example:

```
2 + 2
```

is a command.

It tells R to perform a calculation.

This is also a command:

```
x <- 5
```

It tells R to store the value `5` in an object called `x`.

What Is an R Function?

A **function** is a tool in R that performs a particular task.

Functions usually have a name followed by parentheses.

For example:

```
head()
```

is a function.

The function name is:

```
head
```

The parentheses indicate that we want R to use the function.

We can provide information inside the parentheses:

```
head(winter_smoke)
```

This means:

> Use the `head()` function on the object called `winter_smoke`.

> **i** A function call is also a command
>
> When we enter:
>
> ```
> head(winter_smoke)
> ```
>
> we are giving R a **command**.
> That command contains a call to the **`head()` function**.
> You can think of **command** as the broader term.
> A **function** is one of the tools we can use inside a command.

The Console and the Script Have Different Jobs

During an analysis, we will sometimes run commands simply to investigate the data.

For example:

```
head(winter_smoke)
dim(winter_smoke)
str(winter_smoke)
```

These commands help us answer questions such as:

- Did the data import correctly?
- How many observations are there?
- What variables are present?
- How did R interpret those variables?

We may run these commands directly in the **Console** while we investigate the dataset.

We do not need to save every exploratory command in our final R script.

Our R script should preserve the commands that are important for reproducing the analysis.

For example:

```
winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)

winter_smoke$date_time <- as.POSIXct(
  winter_smoke$date_time
)

plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

Those commands perform actions that we want to reproduce.

> Think of it this way
>
> **Console:** investigate, test, check, and explore.
> **R script:** save the important steps needed to reproduce your analysis.
> You may sometimes decide that an exploratory command is important enough to preserve.
> The goal is not to follow a rigid rule.
> The goal is to understand why you are running each command.

Running Code from Your R Script

Writing code in the Source pane does not automatically run it.

R only performs the command when you send it to the Console.

To run a command from your script:

1. Click on the line you want to run.
2. Select **Run** at the top of the Source pane.

You can also use:

Windows: `Ctrl + Enter`

macOS: `Command + Enter`

The command will appear in the Console and R will execute it.

> ❗ Run your commands in order
>
> R can only use objects that have already been created during the current session.
> For example, this command:
>
> ```
> head(winter_smoke)
> ```
>
> will only work after the object `winter_smoke` has been created.
> If R returns:
>
> ```
> Error: object 'winter_smoke' not found
> ```
>
> ask yourself:
>
> Did I run the earlier command that created `winter_smoke`?

Your First R Commands

We will start very simply.

Enter this command in the Console:

```
2 + 2
```

R should return:

```
[1] 4
```

Try another:

```
10 / 2
```

At its most basic level, working with R means:

> We give R an instruction and R returns something.

Creating an Object

We can also store information in R.

Enter:

```
x <- 5
```

Now ask R what is stored in `x`.

```
x
```

The symbol:

```
<-
```

is called the assignment operator.

You can read:

```
x <- 5
```

as:

> Store the value 5 in an object called `x`.

We will use this idea constantly when working with datasets.

Getting the Dataset

Now that we have a place for our work, we need the dataset we will analyze.

For this lab, we will use:

`winter-smoke-2019-12-27_to_2019-12-31.csv`

The dataset contains ambient particle measurements collected from December 27 through December 31, 2019.

Download the Dataset

The dataset is available in the SHM 377 GitHub repository.

Use the guide below to navigate to the dataset and download the CSV file.

Getting the Dataset from GitHub

For this lab, you will download the monitoring dataset directly from the SHM 377 GitHub repository. Follow the steps below to navigate to the correct folder and download the CSV file.

What you need

Download the file named `winter-smoke-2019-12-27_to_2019-12-31.csv`.
Save it in your project folder (e.g., `virtual-lab-01/`).

Step 1: Open the course repository

Go to the SHM 377 GitHub repository and look for the lectures folder. Click on lectures to open it.

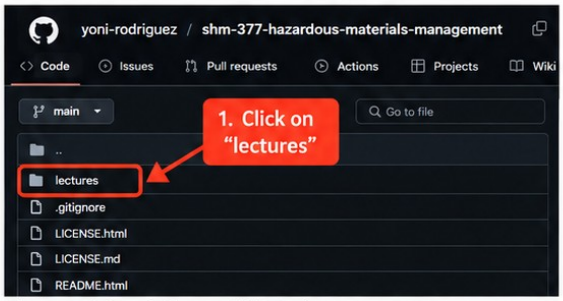


Figure 1. Click on the lectures folder in the course repository.

Step 2: Open week-01 folder

Inside the lectures folder, find and click on the **week-01** folder.

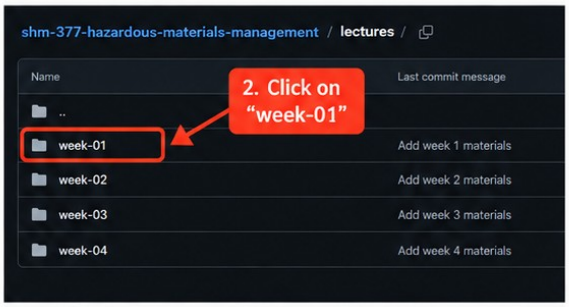


Figure 2. Click on the week-01 folder.

Step 3: Open 01-R-introduction folder

Inside the `week-01` folder, click on the `01-R-introduction` folder.

Figure 3. Click on the `01-R-introduction` folder.

Step 4: Download the CSV file

Inside the `01-R-introduction` folder, find the file named `winter-smoke-2019-12-27_to_2019-12-31.csv`. Click on the file name to open it, then click the Download button.

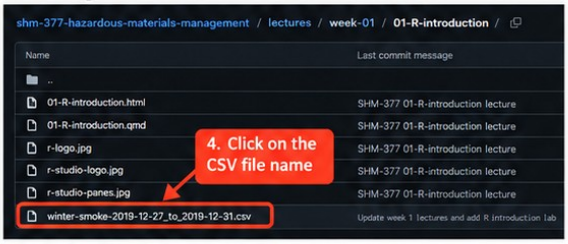


Figure 4. Click on the dataset file to open it.

After downloading

1. Save the CSV file in your `virtual-lab-01/` folder (or another project folder you create).
2. Use R to import it with:

```
winter_smoke <- read.csv("winter-smoke-2019-12-27_to_2019-12-31.csv")
```

Follow the steps shown above:

1. Open the SHM 377 GitHub repository.
2. Open the `lectures` folder.
3. Open the `week-01` folder.
4. Open the `01-R-introduction` folder.

5. Select `winter-smoke-2019-12-27_to_2019-12-31.csv`.
6. Download the CSV file.
7. Save the file inside your `virtual-lab-01` folder.

> **!** Make sure you download the CSV file
>
> The file you need for this lab is:
> `winter-smoke-2019-12-27_to_2019-12-31.csv`
> Do not download the `.qmd`, `.html`, or `.pdf` file when you are trying to obtain the dataset.

Your folder should now contain:

```
SHM-377/
   virtual-lab-01/
       virtual-lab-01.R
       winter-smoke-2019-12-27_to_2019-12-31.csv
```

Where Is R Looking?

Before we ask R to import a file, we need to know where R is looking for that file.

This location is called the **working directory**.

Your RStudio Project and your working directory are related, but they are not the same thing.

The **RStudio Project** organizes the larger body of work.

The **working directory** tells R which folder it should currently use when looking for files.

Check the Working Directory

Use the `getwd()` function.

Function: `getwd()`

Function name: `getwd`

Purpose: Show the current working directory.

The name means:

```
get working directory
```

Enter this command in the **Console**:

```
getwd()
```

R will display the folder it is currently using.

Ask yourself:

Is R looking inside my `virtual-lab-01` folder?

See What Files R Can Find

We can also use:

```
list.files()
```

Function: `list.files()`

Function name: `list.files`

Purpose: Show the files available in the current working directory.

Run:

```
list.files()
```

You should see:

```
winter-smoke-2019-12-27_to_2019-12-31.csv
```

among the files listed.

If you can see the CSV in `list.files()`, R can see the file.

Set the Working Directory

If `getwd()` does not show your `virtual-lab-01` folder, set the working directory before continuing.

Because your R script is saved inside `virtual-lab-01`, the easiest method is:

1. Make sure `virtual-lab-01.R` is open in the Source pane.
2. Select **Session**.
3. Select **Set Working Directory**.
4. Select **To Source File Location**.

R should now use the folder containing your **R script.**

Check again:

```
getwd()
```

Then:

```
list.files()
```

You should now be able to see the CSV file.

❗ This is one of the first things to check when R cannot find a file

If R returns an error such as:

```
cannot open file
```

or:

```
No such file or directory
```

check:

```
getwd()
list.files()
```

Ask:

1. Where is R looking?
2. Is the file actually there?

This simple check can solve many file import problems.

Importing the Dataset

Now we are ready to import the dataset.

We will save this command in our **R script** because importing the data is an important part of reproducing the analysis.

The function we will use is `read.csv()`.

Function: `read.csv()`

Function name: `read.csv`

Purpose: Import data from a CSV file.

Enter this command in your **R script:**

```
winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)
```

Run the command.

Read it from right to left.

First:

```
read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)
```

tells R to read the CSV file.

Then:

```
winter_smoke <-
```

stores the imported dataset in an object called `winter_smoke`.

R will:

1. find the CSV file
2. read the file
3. create a dataset in R
4. store the dataset under the name `winter_smoke`

After the command runs successfully, you should see `winter_smoke` appear in the **Environment** pane.

Inspect Before You Analyze

Now we want to investigate the dataset before deciding what needs to be changed or analyzed.

These next commands are primarily **exploratory**.

You can run them in the **Console.**

You do not need to save every one in your R script.

Our questions are:

- Did the file import correctly?
- What variables are present?
- How many observations are there?
- What does one row represent?
- Did R interpret the variables correctly?
- Do the values look reasonable?

Look at the First Rows

The `head()` function shows the first several rows.

Function: `head()`

Function name: `head`

Purpose: Look at the first several rows of a dataset.

Run in the **Console:**

```
head(winter_smoke)
```

Ask yourself:

> What does one row appear to represent?

Look at the Last Rows

The `tail()` function shows the last several rows.

Function: `tail()`

Function name: `tail`

Purpose: Look at the last several rows of a dataset.

Run in the **Console:**

```
tail(winter_smoke)
```

Looking at both the beginning and end can help us confirm that the file contains the time period we expected.

Variable Names

The `names()` function shows the variable names.

Function: `names()`

Function name: `names`

Purpose: Show the variable names in a dataset.

Run in the **Console:**

```
names(winter_smoke)
```

We expect:

```
date_time
concentration
```

Dataset Dimensions

The `dim()` function shows the dimensions of an object.

Function: `dim()`

Function name: `dim`

Purpose: Show the number of rows and columns in a dataset.

The name `dim` is short for **dimensions**.

Run in the **Console:**

```
dim(winter_smoke)
```

R reports two numbers in this order:

```
rows columns
```

The rows represent observations.

The columns represent variables.

For this dataset, each row represents one monitoring measurement.

Examine the Structure

The `str()` function shows the structure of an object.

Function: `str()`

Function name: `str`

Purpose: Examine the structure of a dataset.

The name `str` is short for **structure**.

Run in the **Console:**

```
str(winter_smoke)
```

This function tells us several things at once, including:

- variable names
- variable types
- example values

This is one of the most useful functions to run immediately after importing a dataset.

Did R Understand the Date and Time?

Our inspection revealed something important.

R may have imported the `date_time` variable as text rather than as date and time data.

The exploratory commands helped us identify a problem.

Now we need to make a change to the dataset.

Because this change is part of our analysis, we **will save it in the R script**.

The function we will use is:

```
as.POSIXct()
```

Function: `as.POSIXct()`

Function name: `as.POSIXct`

Purpose: Convert date and time information into a format that R recognizes as date and time data.

The beginning of the function name gives us a useful clue:

```
as.
```

Functions beginning with `as.` are often used to convert something from one data type into another.

For our purposes, think of:

```
as.POSIXct()
```

as:

> Convert this information into date and time data that R can understand.

You do not need to memorize the name `as.POSIXct()`.

Add this command to your **R script:**

```
winter_smoke$date_time <- as.POSIXct(
  winter_smoke$date_time
)
```

Run the command.

Read it from right to left.

First:

```
winter_smoke$date_time
```

identifies the `date_time` variable inside `winter_smoke`.

Then:

```
as.POSIXct(winter_smoke$date_time)
```

asks R to convert those values into date and time data.

Finally:

```
winter_smoke$date_time <-
```

stores the converted values back into the `date_time` variable.

Now return to the **Console** and check again:

```
str(winter_smoke)
```

Did the variable type change?

Notice the workflow:

```
Inspect in the Console
        ↓
Identify a problem
        ↓
Write the needed change in the R script
        ↓
Run the change
        ↓
Check again in the Console
```

This introduces an important habit:

> **Do not assume your code worked. Check.**

Check the Measurements

Before calculating averages or making graphs, inspect the range of the measurements.

These are also useful exploratory checks.

Minimum

The `min()` function finds the smallest value.

Function: `min()`

Function name: `min`

Purpose: Find the minimum value.

Run in the **Console:**

```
min(winter_smoke$concentration)
```

Maximum

The `max()` function finds the largest value.

Function: `max()`

Function name: `max`

Purpose: Find the maximum value.

Run in the **Console:**

```
max(winter_smoke$concentration)
```

The observed concentration range in this teaching dataset is approximately:

```
0.000 to 0.288
```

At this stage, ask:

> Do these measurements appear plausible?

Unusual Values

Suppose one concentration is much higher than most of the others.

Should we immediately delete it?

No.

An unusual value could represent:

- a real environmental event
- a nearby source
- unusual instrument behavior
- a processing problem
- something we do not yet understand

Before removing a value, ask:

- Is it physically possible?
- Do nearby observations show a similar pattern?
- Was something happening at that time?
- Could the instrument have malfunctioned?
- Do we have enough evidence to make a decision?

Unusual does not automatically mean incorrect.

Describe the Data

Now that we have inspected and checked the dataset, we can begin summarizing it.

Whether a descriptive statistic belongs in your final script depends on whether it is part of the analysis you intend to report.

For this lab, we will calculate several summaries.

Mean

The `mean()` function calculates the arithmetic mean.

Function: `mean()`

Function name: `mean`

Purpose: Calculate the mean of a set of values.

```
mean(winter_smoke$concentration)
```

The mean gives us one way to describe the center of the measurements.

But one number cannot show us how concentrations changed through time.

Median

The `median()` function calculates the median.

Function: `median()`

Function name: `median`

Purpose: Calculate the median of a set of values.

```
median(winter_smoke$concentration)
```

Compare the mean and median.

Are they similar?

If not, what might that suggest about the distribution of measurements?

Summary

The `summary()` function produces several summary statistics at once.

Function: `summary()`

Function name: `summary`

Purpose: Produce a summary of an object.

```
summary(winter_smoke$concentration)
```

For a numeric variable, R provides several descriptive statistics.

Look at the output.

What information does R provide?

What information is still missing?

Descriptive Statistics Have Limits

A numerical summary can tell us about:

- the center
- the range
- the distribution

But it cannot tell us when concentrations changed.

For that, we need to look at the measurements through time.

Visualizing Concentration Through Time

Because these measurements were repeatedly collected over several days, a time series graph is a natural first visualization.

We will place:

time on the x axis

and

particle concentration on the y axis

These measurements were collected using a TSI DustTrak.

The particle concentration measurements are reported in:

milligrams per cubic meter (mg/m^3)

The function we will use is `plot()`.

Function: `plot()`

Function name: `plot`

Purpose: Create a graph.

This is a command that we want to preserve because the graph is part of our analysis.

Add it to your R script:

```
plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

Run the command.

The graph will appear in the **Plots** pane in RStudio.

Reading the Plot Function

The `plot()` function is more complicated than the functions we have used so far.

That makes it a useful example for learning how to read R code.

Look at the command again:

```
plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

Everything inside the parentheses provides information to the `plot()` function.

These pieces of information are called **arguments**.

Arguments

An **argument** gives a function information it needs to perform its task.

Our `plot()` function contains five arguments:

```
winter_smoke$date_time
winter_smoke$concentration
type = "l"
xlab = "Date / Time"
ylab = "Particle Concentration (mg/m³)"
```

Each argument is separated by a comma.

Why Do We Use Commas?

Inside a function, commas separate one argument from the next.

For example:

```
plot(
  winter_smoke$date_time,
  winter_smoke$concentration
)
```

The comma tells R that:

```
winter_smoke$date_time
```

is one argument and:

```
winter_smoke$concentration
```

is another argument.

We do not use a semicolon here.

A semicolon can be used in R to separate complete commands or expressions.

For example:

```
x <- 5; x
```

contains two complete instructions.

That is different from separating arguments inside a function.

For this course, use **commas to separate arguments inside functions**.

Why Does `date_time` Come First?

The first two arguments in our `plot()` function are:

```
winter_smoke$date_time,
winter_smoke$concentration
```

The `plot()` function expects information about what should appear on the x axis and what should appear on the y axis.

When we provide arguments without writing their names, R uses their position.

The first argument is used for the x axis.

The second argument is used for the y axis.

Therefore:

```
winter_smoke$date_time
```

comes first because we want time on the x axis.

And:

```
winter_smoke$concentration
```

comes second because we want particle concentration on the y axis.

Positional Arguments

The first two arguments are being identified by their position.

These are called **positional arguments**.

We could make their meaning more explicit by writing:

```
plot(
  x = winter_smoke$date_time,
  y = winter_smoke$concentration
)
```

This tells R the same thing.

Named Arguments

The remaining arguments are:

```
type = "l",
xlab = "Date / Time",
ylab = "Particle Concentration (mg/m³)"
```

These are **named arguments**.

Instead of relying on position, we explicitly tell R which setting we are changing.

What Does `type = "l"` Mean?

```
type = "l"
```

The `type` argument controls how the observations are drawn.

The letter `"l"` stands for **line**.

This tells R to connect the observations with a line.

What Does `xlab` Mean?

```
xlab = "Date / Time"
```

`xlab` means **x axis label**.

What Does `ylab` Mean?

```
ylab = "Particle Concentration (mg/m³)"
```

`ylab` means **y axis label**.

The unit is part of the label because the numbers alone do not tell us what was measured or how the measurement should be interpreted.

What Does = Mean Inside the Function?

Earlier we used:

```
<-
```

to store information in an object.

Inside a function, we often use:

```
=
```

to provide a value for a named argument.

For example:

```
type = "l"
```

means:

> Use `"l"` for the `type` argument.

Exporting Your Plot

Creating the graph in R is only part of the job.

You may need to use the graph in:

- a report
- a Word document
- a presentation
- another communication product

RStudio allows us to export the graph from the **Plots** pane.

After creating the graph:

1. Open the **Plots** pane.
2. Select **Export**.
3. Choose **Save as Image** or **Save as PDF**.
4. Choose where you want to save the file.
5. Give the figure a descriptive file name.
6. Save the figure.

For example:

```
winter-smoke-concentration-time-series.jpg
```

You can then insert the saved figure into Word, PowerPoint, or another document.

> Save figures with the rest of your project
>
> A useful project structure might eventually look like:
>
> ```
> SHM-377/
>   virtual-lab-01/
>       winter-smoke-2019-12-27_to_2019-12-31.csv
>       virtual-lab-01.R
>       winter-smoke-concentration-time-series.jpg
> ```
>
> Keeping the data, code, and figures together makes it easier to understand where the products of your analysis came from.

What Should Be in Your R Script?

By this point, your R script does not need to contain every command that you entered while exploring the dataset.

A simple version of the important reproducible steps might look like:

```r
# Virtual Lab 01
# Your Name
# Date

# Import data

winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)

# Convert date and time

winter_smoke$date_time <- as.POSIXct(
  winter_smoke$date_time
)

# Plot concentration through time

plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

During the analysis, you may also have used:

```r
getwd()
list.files()
head(winter_smoke)
tail(winter_smoke)
names(winter_smoke)
dim(winter_smoke)
str(winter_smoke)
min(winter_smoke$concentration)
max(winter_smoke$concentration)
```

Those commands helped you **evaluate and understand the dataset**.

They do not all need to remain in the final script unless you decide they are important to the analysis you want to preserve.

The important distinction is:

> **Use the Console to investigate. Use the script to preserve the analysis you want to reproduce.**

Why Units Matter

A number without a unit can be difficult or impossible to interpret correctly.

In this dataset, the concentration values came from a TSI DustTrak and are reported in:

milligrams per cubic meter (mg/m³)

However, notice that our CSV variable is simply called:

```
concentration
```

The variable name itself does not tell us the unit.

This is common when working with environmental monitoring data.

A CSV file may contain the numerical measurements but may not contain all of the information needed to understand those measurements.

That information may instead come from:

- instrument settings
- field notes
- data dictionaries
- project documentation
- sampling protocols
- laboratory reports
- instrument manuals
- knowledge of the monitoring method

This means that importing a dataset is not enough.

We also need to understand:

> **What was measured, how was it measured, and what units were used?**

The appropriate unit depends on the agent, the environmental medium, the instrument, and the measurement method.

For example, an air concentration may be expressed differently from a concentration measured in water or soil.

Different agents may also be reported using different units.

Understanding the standard measurement practices for the **medium and agent being evaluated** is part of understanding the dataset.

> **!** Never assume the units
>
> Do not infer a unit only from the numerical values in a dataset.
> Confirm the units using the instrument information, project documentation, sampling method, or another reliable source.
> If the unit cannot be verified, do not present one as fact.

For this dataset, we have verified the measurement information, so our graph should identify the y axis as:

```
Particle Concentration (mg/m³)
```

Including the unit makes the graph more informative and preserves important measurement context.

Look Before Explaining

Study the graph before trying to explain it.

Ask:

- Is concentration constant?
- Are there peaks?
- When do the largest peaks occur?
- Are there periods of relatively low concentration?
- Are some periods more variable?
- Do any patterns appear to repeat?

The first step is observation.

Observation and Interpretation

A statement such as:

> Particle concentration varied throughout the monitoring period, with periods of relatively low concentration interrupted by higher measured concentrations.

is an observation based on the dataset.

A statement such as:

> Residential wood burning caused the peaks.

is a source attribution claim.

The dataset alone does not prove that.

What Can We Conclude?

From the measurements we can examine:

- when concentrations increased
- how high measured concentrations became
- how concentrations changed through time
- whether elevated periods persisted
- how variable the measurements were

What Can We Not Conclude?

From this dataset alone, we cannot determine:

- the exact source of every particle
- why every concentration increase occurred
- whether one source caused a particular peak

Additional information would be needed.

What Other Data Could Help?

The monitoring results may lead us to new questions.

For example:

- What was the wind speed?
- What was the wind direction?
- What was the temperature?
- Were there atmospheric inversions?
- Were there nearby emission sources?
- Did local heating activity change?

This is how data analysis often works.

One answer produces better questions.

Return to the Original Question

Our original question was:

> **What happened to ambient particle concentrations during five days of winter monitoring?**

We now have several ways to answer that question.

We:

- inspected the dataset
- checked its structure
- examined its measurement range
- calculated descriptive statistics
- plotted concentration through time
- interpreted the observed pattern

The SHM 377 Data Workflow

Throughout this course, we will repeatedly use a workflow like this:

```
Question
   ↓
Data
   ↓
Check
   ↓
Describe / Analyze
   ↓
Visualize
   ↓
Interpret
   ↓
Communicate / Manage
```

The specific commands will change.

The reasoning process will remain similar.

Connecting Data to Hazardous Materials Management

In hazardous materials management, we may begin with questions about:

```
Material
   ↓
Properties
   ↓
Source / Activity
   ↓
Release
   ↓
Environmental Medium
   ↓
Transport
   ↓
```

```
Exposure
   ↓
Dose
   ↓
Effects
```

Monitoring provides evidence about parts of that system.

Data analysis helps us move from measurements toward evaluation and decision making.

```
Hazardous Materials Problem
          ↓
      Measurement
          ↓
         Data
          ↓
      Evaluation
          ↓
    Interpretation
          ↓
 Management / Control
```

Reproducibility

One reason we are using R is that the analysis itself can be saved.

For example:

```
winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)

head(winter_smoke)

summary(winter_smoke$concentration)

plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

These commands provide a record of what we did.

Someone can inspect the code and understand how the analysis was produced.

Later, this idea will lead naturally into Quarto reports.

You Do Not Need to Memorize Everything

Today you encountered commands such as:

```
read.csv()
head()
tail()
names()
dim()
str()
as.POSIXct()
min()
max()
mean()
median()
summary()
plot()
```

You do not need to memorize all of these immediately.

What matters more is understanding why we use them.

Ask yourself:

```
What is my question?

What does my dataset contain?

Did the data import correctly?

Do the measurements look reasonable?

How can I describe them?

How can I visualize them?

What does the evidence show?

What can I not conclude?
```

What You Accomplished

In this first virtual lab, you:

- worked with a real environmental monitoring dataset
- imported data into R
- inspected the dataset
- checked the measurements
- calculated basic descriptive statistics
- created a time series visualization
- interpreted the results
- distinguished measurement from source attribution

The Main Idea

> **We are not learning R for the sake of learning R.**

We are learning how to use data to investigate, evaluate, and communicate hazardous materials problems.

Where We Go Next

We will continue building on the same workflow throughout SHM 377.

Over time, we will move from basic R commands toward:

```
R
↓
R scripts
↓
data analysis
↓
visualization
↓
interpretation
↓
reproducible Quarto reports
```

We do not need to learn everything at once.

We start with the question.

Then we use the data to work toward an answer.
